

SOLID原则与GoF设计模式在C语言中的实现

C语言没有类、继承和虚方法，但每一条SOLID原则和许多GoF设计模式 都可以通过结构体、函数指针和不透明类型来实现。本文档将逐一说明。

第一部分：C语言中的SOLID原则

S — SRP（单一职责原则）

每个模块（.c/.h文件对）只有一个变更理由。如果一个模块同时处理 UART通信和协议解析，就应该拆分为两个模块。

测试方法：能否用一句不包含"和"字的话描述该模块的用途？如果不能，说明它承担了太多职责。

```
/* 反面示例 - 一个模块做所有事 */
comm_module.c
├─ UART硬件初始化
├─ UART收发
├─ 协议帧解析
├─ CRC计算
└─ 命令分发

/* 正面示例 - 职责分离 */
uart_driver.c      → UART硬件抽象
protocol_parser.c  → 帧解析和校验
crc.c              → CRC计算
command_handler.c  → 命令分发和执行
```

O — OCP（开闭原则）

模块对扩展开放，对修改关闭。在C语言中，通过函数指针表（vtable）和注册模式来实现。

```
/* 协议处理器 - 新增命令无需修改现有代码 */
```

```

typedef int (*CmdHandler_t)(const uint8_t *payload,
                           uint16_t len);

typedef struct {
    uint8_t      cmd_id;
    CmdHandler_t handler;
} CmdEntry_t;

/* 新命令通过添加到表中实现，现有处理函数不需修改 */
static const CmdEntry_t CMD_TABLE[] = {
    { CMD_READ_STATUS,  handle_read_status },
    { CMD_SET_CONFIG,   handle_set_config   },
    { CMD_FIRMWARE_VER, handle_fw_version   },
    /* 在此添加新命令 - 无需修改现有代码 */
};

int dispatch_command(uint8_t cmd_id,
                    const uint8_t *payload,
                    uint16_t len)
{
    for (uint32_t i = 0; i < CMD_TABLE_SIZE; i++) {
        if (CMD_TABLE[i].cmd_id == cmd_id) {
            return CMD_TABLE[i].handler(payload, len);
        }
    }
    return ERR_UNKNOWN_CMD;
}

```

L — LSP（里氏替换原则）

函数指针接口背后的任何实现都必须可以替换，而调用者无需知道 具体使用的是哪个实现。所有实现必须遵守相同的契约（前置条件、后置条件、错误码）。

```

/* 抽象存储接口 */
typedef struct {
    int (*read)(void *ctx, uint32_t addr,
               uint8_t *buf, uint16_t len);
    int (*write)(void *ctx, uint32_t addr,
               const uint8_t *buf, uint16_t len);
    int (*erase)(void *ctx, uint32_t addr,
               uint32_t size);
} StorageOps_t;

/* 两种实现都遵守相同的契约：
 * - 成功返回 ERR_OK

```

```

* - 无效地址/长度返回 ERR_INVALID_PARAM
* - 硬件故障返回 ERR_HW_FAULT
* - 不会修改指定范围之外的数据 */

/* 内部Flash实现 */
static const StorageOps_t flash_ops = {
    .read  = flash_read,
    .write = flash_write,
    .erase = flash_erase,
};

/* 外部EEPROM实现 */
static const StorageOps_t eeprom_ops = {
    .read  = eeprom_read,
    .write = eeprom_write,
    .erase = eeprom_erase,
};

/* 调用者不知道使用的是哪种存储 */
int save_config(const StorageOps_t *ops, void *ctx,
               const Config_t *cfg)
{
    return ops->write(ctx, CFG_ADDR,
                    (const uint8_t *)cfg,
                    sizeof(Config_t));
}

```

I — ISP（接口隔离原则）

不要强迫模块依赖它不使用的接口。将臃肿的接口拆分为聚焦的小接口。

完整示例参见 `01_架构设计.md`。核心规则：如果使用者只需要 读取功能，就只给它只读接口，而非完整的读写擦除接口。

D — DIP（依赖倒置原则）

高层模块不能依赖底层模块。两者都应依赖抽象（函数指针接口）。

```

/* 反面示例 - 高层依赖底层 */
app_logger.c → #include "spi_flash.h"
               spi_flash_write(addr, data, len);

/* 正面示例 - 两者都依赖抽象 */
app_logger.c → 使用 StorageOps_t 接口
spi_flash.c  → 实现 StorageOps_t 接口

```

```
/* Logger完全不知道SPI Flash的存在。  
 * 测试时可以替换为基于RAM的Mock实现。 */
```

C语言中的依赖注入：

```
typedef struct {  
    const StorageOps_t *storage;  
    void                *storage_ctx;  
    /* ... 其他依赖 ... */  
} Logger_t;  
  
Logger_t *Logger_Create(const StorageOps_t *storage,  
                        void *storage_ctx)  
{  
    Logger_t *self = platform_malloc(sizeof(Logger_t));  
    if (self == NULL) {  
        return NULL;  
    }  
    self->storage      = storage;  
    self->storage_ctx = storage_ctx;  
    return self;  
}
```

第二部分：嵌入式C语言中的GoF设计模式

并非所有23种GoF模式都适用于嵌入式C。以下是固件开发中最常用的 模式，按使用频率排列。

状态机模式 (State)

嵌入式系统中最重要模式。封装与状态相关的行为和状态转换。

```
/* —— 基于函数指针表的状态机 —— */  
  
typedef enum {  
    STATE_IDLE,  
    STATE_CONNECTING,  
    STATE_ACTIVE,  
    STATE_ERROR,  
    STATE_COUNT  
} DevState_t;
```

```

typedef enum {
    EVT_START,
    EVT_CONNECTED,
    EVT_DATA,
    EVT_FAULT,
    EVT_RESET,
    EVT_COUNT
} DevEvent_t;

typedef DevState_t (*StateHandler_t)(
    Device_t *self, DevEvent_t evt);

/* 每个状态都有自己的处理函数 */
static DevState_t state_idle(Device_t *self,
                              DevEvent_t evt);
static DevState_t state_connecting(Device_t *self,
                                    DevEvent_t evt);
static DevState_t state_active(Device_t *self,
                                DevEvent_t evt);
static DevState_t state_error(Device_t *self,
                               DevEvent_t evt);

static const StateHandler_t STATE_TABLE[STATE_COUNT] = {
    [STATE_IDLE]      = state_idle,
    [STATE_CONNECTING] = state_connecting,
    [STATE_ACTIVE]    = state_active,
    [STATE_ERROR]     = state_error,
};

void Device_HandleEvent(Device_t *self, DevEvent_t evt)
{
    ASSERT(self != NULL);
    ASSERT(self->state < STATE_COUNT);

    DevState_t next = STATE_TABLE[self->state](
        self, evt);

    if (next != self->state) {
        on_state_exit(self, self->state);
        self->state = next;
        on_state_enter(self, next);
    }
}

```

观察者模式（发布-订阅）

解耦事件生产者和消费者。是固件中通知系统的核心模式。

```
#define MAX_OBSERVERS (8U)

typedef void (*ObserverCb_t)(void *ctx,
                             uint32_t event_id,
                             const void *data);

typedef struct {
    ObserverCb_t cb;
    void *ctx;
} Observer_t;

typedef struct {
    Observer_t observers[MAX_OBSERVERS];
    uint8_t count;
    os_mutex_t mutex; /* 多线程环境下保护观察者列表 */
} Subject_t;

void Subject_Init(Subject_t *self)
{
    self->count = 0;
    os_mutex_init(&self->mutex);
}

int Subject_Subscribe(Subject_t *self,
                     ObserverCb_t cb, void *ctx)
{
    os_mutex_lock(&self->mutex);
    if (self->count ≥ MAX_OBSERVERS) {
        os_mutex_unlock(&self->mutex);
        return ERR_BUFFER_FULL;
    }
    self->observers[self->count].cb = cb;
    self->observers[self->count].ctx = ctx;
    self->count++;
    os_mutex_unlock(&self->mutex);
    return ERR_OK;
}

void Subject_Notify(const Subject_t *self,
                  uint32_t event_id,
                  const void *data)
```

```

{
    /* 注意：如果在Notify过程中不会有Subscribe/Unsubscribe
    * 操作（如仅在初始化阶段注册），可以省略加锁。
    * 如果可能并发修改观察者列表，则需要加锁保护。 */
    for (uint8_t i = 0; i < self->count; i++) {
        if (self->observers[i].cb != NULL) {
            self->observers[i].cb(
                self->observers[i].ctx,
                event_id, data);
        }
    }
}
}

```

观察者模式的线程安全注意事项

1. **Subscribe/Unsubscribe** 必须加锁保护观察者列表
2. **Notify** 遍历过程中如果观察者列表可能被修改，也需要加锁 或使用快照（先复制列表再遍历）
3. 回调函数内部不要再调用 Subscribe/Unsubscribe，否则会死锁
4. 如果在 ISR 中触发 Notify，不能使用互斥锁——应改用消息队列 将通知投递到任务上下文处理
5. 回调执行时间应尽量短，避免阻塞其他观察者的通知

策略模式 (Strategy)

通过函数指针在运行时切换算法。适用于可配置的协议、压缩、加密等场景。

```

typedef struct {
    uint32_t (*calc)(const uint8_t *data,
                     uint32_t len);
    const char *name;
} ChecksumStrategy_t;

static const ChecksumStrategy_t CRC32_STRATEGY = {
    .calc = crc32_calculate,
    .name = "CRC32",
};

static const ChecksumStrategy_t XOR_STRATEGY = {
    .calc = xor_checksum,
    .name = "XOR",
};

```

```

typedef struct {
    const ChecksumStrategy_t *checksum;
    /* ... */
} Protocol_t;

/* 调用者在初始化时选择策略 */
Protocol_t *Protocol_Create(
    const ChecksumStrategy_t *cs)
{
    Protocol_t *self = platform_malloc(
        sizeof(Protocol_t));
    if (self == NULL) {
        return NULL;
    }
    self->checksum = cs;
    return self;
}

```

工厂模式 (Factory)

集中管理对象创建。当具体类型在运行时确定时非常有用（例如基于硬件检测）。

```

typedef struct Sensor Sensor_t;

/* 抽象传感器接口 */
typedef struct {
    int (*init)(Sensor_t *self);
    int (*read)(Sensor_t *self, int32_t *value);
    void (*deinit)(Sensor_t *self);
} SensorOps_t;

/* 工厂函数 */
Sensor_t *Sensor_Create(SensorType_t type)
{
    switch (type) {
        case SENSOR_TYPE_TEMP_NTC:
            return NtcSensor_Create();
        case SENSOR_TYPE_TEMP_PT100:
            return Pt100Sensor_Create();
        case SENSOR_TYPE_PRESSURE:
            return PressureSensor_Create();
        default:
            return NULL;
    }
}

```



```
}  
}
```

单例模式 (Singleton)

确保只有一个实例存在。在嵌入式C中，使用文件作用域的静态实例 加访问函数。在多线程环境中避免延迟初始化——在启动时完成初始化。

```
/* 在 system_config.c 中 */  
static SystemConfig_t s_instance;  
static bool s_initialized = false;  
  
int SystemConfig_Init(const ConfigParams_t *params)  
{  
    if (s_initialized) {  
        return ERR_ALREADY_INIT;  
    }  
    /* 初始化 s_instance ... */  
    s_initialized = true;  
    return ERR_OK;  
}  
  
SystemConfig_t *SystemConfig_GetInstance(void)  
{  
    ASSERT(s_initialized);  
    return &s_instance;  
}
```

适配器模式 (Adapter)

将现有接口包装为所需的接口。在集成第三方库或遗留代码时非常常见。

```
/* 旧传感器返回原始ADC值 */  
uint16_t legacy_sensor_read_adc(void);  
  
/* 新系统需要 SensorOps_t 接口 */  
static int adapted_read(Sensor_t *self,  
                        int32_t *value)  
{  
    uint16_t raw = legacy_sensor_read_adc();  
    *value = (int32_t)raw * SCALE_FACTOR / DIVISOR;  
    return ERR_OK;  
}
```

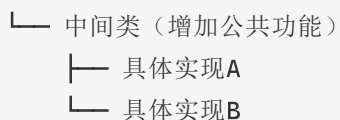
```
static const SensorOps_t LEGACY_ADAPTER = {
    .init    = adapted_init,
    .read    = adapted_read,
    .deinit  = adapted_deinit,
};
```

第三部分：OOP高级模式

多层继承

当需要更复杂的继承体系时，可以建立多层继承链。后续会有配套视频详细讲解，B站搜索「兆鸣嵌入式」观看。

基类（抽象接口）



示例：ADC采集驱动

adc_channel_base_t（抽象基类：定义start/stop/read接口）

- └─ adc_channel_ls_t（低速ADC中间类：增加定时采样+环形缓冲）
 - └─ adc_channel_ls_ad7689_t（AD7689芯片实现）
 - └─ adc_channel_ls_ltc2488_t（LTC2488芯片实现）

每一层都有自己的虚函数表（ops），基类提供统一接口，中间类增加通用功能，具体实现对接硬件。

```
/* 抽象基类 */
typedef struct {
    const struct adc_channel_ops *ops;
    const char *name;
} adc_channel_base_t;

/* 中间类 - 增加低速采样通用功能 */
typedef struct {
    adc_channel_base_t base;          /* 继承基类 */
    ring_buffer_t       sample_buf; /* 通用：环形缓冲 */
    os_timer_t          timer;       /* 通用：定时采样 */
    uint32_t             interval_ms;
```

```

} adc_channel_ls_t;

/* 具体实现 - AD7689芯片 */
typedef struct {
    adc_channel_ls_t    ls;           /* 继承中间类 */
    SPI_HandleTypeDef *spi;          /* 芯片特有 */
    uint8_t             cs_pin;
} adc_channel_ls_ad7689_t;

```

两种向下转型模式

模式A: pderived_data指针 - 基类结构体中保存一个 `void *pderived_data` 指向派生类 - 适合不想修改基类结构体布局的场景

```

typedef struct {
    const struct device_ops *ops;
    void *pderived_data; /* 指向派生类实例 */
} device_base_t;

/* 向下转型 */
my_device_t *dev = (my_device_t *)base->pderived_data;

```

模式B: container_of宏 - 派生类将基类嵌入为第一个成员 - 通过 `container_of` 从基类指针计算派生类指针 - 更常用，类型安全性更好

```

/* 向下转型 */
my_device_t *dev = container_of(
    base_ptr, my_device_t, base);

```

详细的 `container_of` 使用方式参见 `01_架构设计.md`。

第四部分：层级状态机与事件驱动架构

层级状态机 (HSM)

简单状态机 (Flat State Machine) 适合状态较少的场景。当状态数量增多且存在共同行为时，应使用层级状态机 (Hierarchical State Machine)：

- 状态可以嵌套（父状态包含子状态）
- 子状态未处理的事件自动传递给父状态

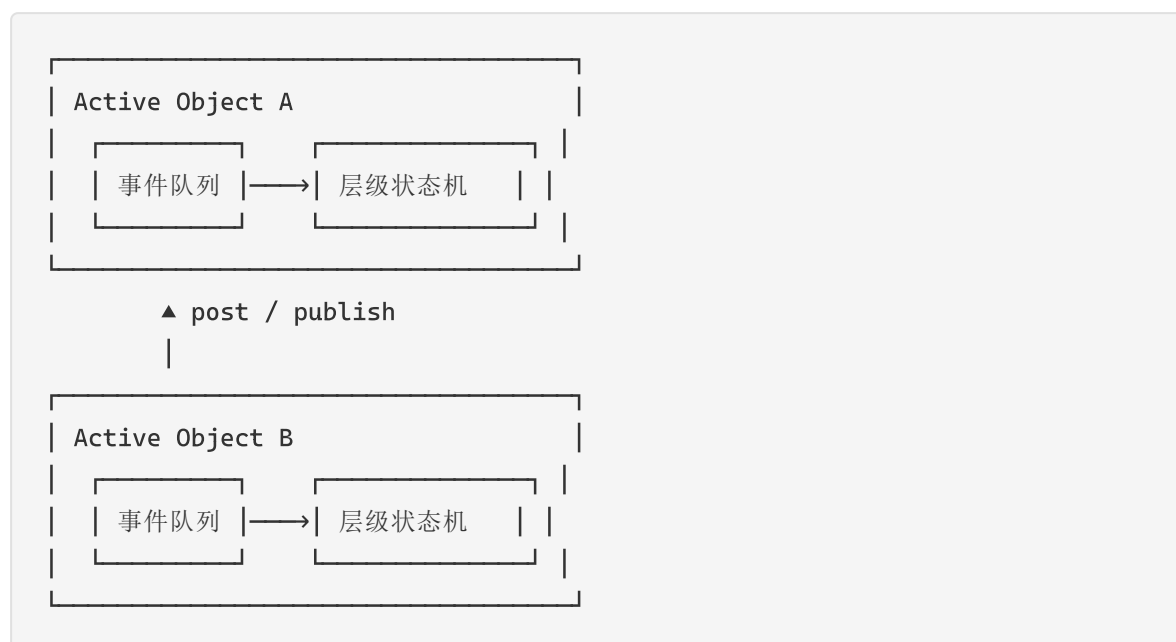
- 每个状态有 entry/exit 动作，状态转换时自动执行
- 大幅减少重复的事件处理代码

推荐框架：**QPC**（Quantum Platform for C），开源，适合嵌入式实时系统。后续会有配套视频详细讲解，B站搜索「兆鸣嵌入式」观看。

事件驱动架构

HSM通常配合事件驱动的 Active Object 模式使用：

- 每个 Active Object 拥有独立的事件队列和线程
- 对象之间通过发布-订阅（publish-subscribe）或 直接投递（post）通信
- 支持紧急事件 LIFO 投递
- 天然线程安全——每个对象的状态机在自己的线程中运行



表驱动的事件订阅

当外部协议命令需要映射到内部事件时，使用表驱动配置：

```

typedef struct {
    const char *cmd_str;
    uint32_t    event_signal;
} event_subscriber_t;

static const event_subscriber_t SUBSCRIBE_TABLE[] = {
    {"CMD_START",  EVENT_SCAN_START},
    {"CMD_STOP",   EVENT_SCAN_STOP},
    {"CMD_STATUS", EVENT_STATUS_QUERY},

```

```
/* 新增命令只需加一行，不改已有代码 */
};
```

命令字符串直接映射到事件信号，通过配置表实现解耦，符合开闭原则。

第五部分：模式选择指南

场景	推荐模式
行为随模式/阶段变化	状态机
状态多且有共同行为	层级状态机（HSM）
多个模块需要事件通知	观察者
算法因配置而异	策略
对象类型在运行时确定	工厂
全局共享资源，仅一个实例	单例
包装不兼容的接口	适配器
扩展行为而不修改代码	OCP + 表驱动
解耦高层/底层模块	DIP + 函数指针
多层级硬件抽象	多层继承 + container_of
线程安全的事件驱动	Active Object + HSM

应避免的反模式

- 上帝模块** — 一个模块做所有事。应用SRP拆分。
- 意大利面条式依赖** — 每个模块都引用其他所有模块。应用DIP解耦。
- 基本类型偏执** — 传递10个原始整数而非结构体。应将相关数据组合为有意义的类型。
- 复制粘贴复用** — 复制代码而非提取共享函数。应用DRY原则。
- 魔法Switch** — 无限增长的巨型switch语句。应用OCP并使用表驱动分发。

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版